

By: Shoumik, Moneba, Judy, Ekam & Abrar

Group Project: Team AB
FreshRoute System Design Document

Team Members:

Name	Email	Student ID
Moneba Hanifi	moneba@my.yorku.ca	219721810
Shoumik Ahmed	shoumik@my.yorku.ca	219881598
Judy El Ghargomi	jelg21@my.yorku.ca	220227237
Ekambir Momi	ekam727@my.yorku.ca	219424407
Abrar Jawad Tarafder	abrart03@my.yorku.ca	219453497

3311 Section M Term W
Class Instructor: Ilir Dema
March 4, 2026

Table of Contents

Cover Page 1
Table Of Contents 2

Chapter 1: Document Overview

Introduction 3

Chapter 2: System Overview and Design

Architecture Diagram 4
Component Design CRC Cards 5
Assumption and Future Work 9

Chapter 1: Document Overview

INTRODUCTION

This document describes the system design for **FreshRoute**, a web-based grocery platform focused on organic and ethically sourced food products. The purpose of this document is to present the overall system structure, the technologies used in its implementation, and the responsibilities of the major components.

FreshRoute follows a **MERN stack architecture**, comprising a React frontend, a Node.js and Express backend, and a MongoDB database accessed via Mongoose. Communication between the frontend and backend occurs through **RESTful APIs using HTTP and JSON**, with Cross-Origin Resource Sharing (CORS) enabled during development.

The backend follows a **Model-View-Controller (MVC)** design pattern. Express routers define API endpoints, controllers handle incoming requests and application logic, and models define the MongoDB schemas and manage database interactions. The backend configuration is handled in **app.js**, while **server.js** serves as the application entry point, establishing the database connection and starting the HTTP server.

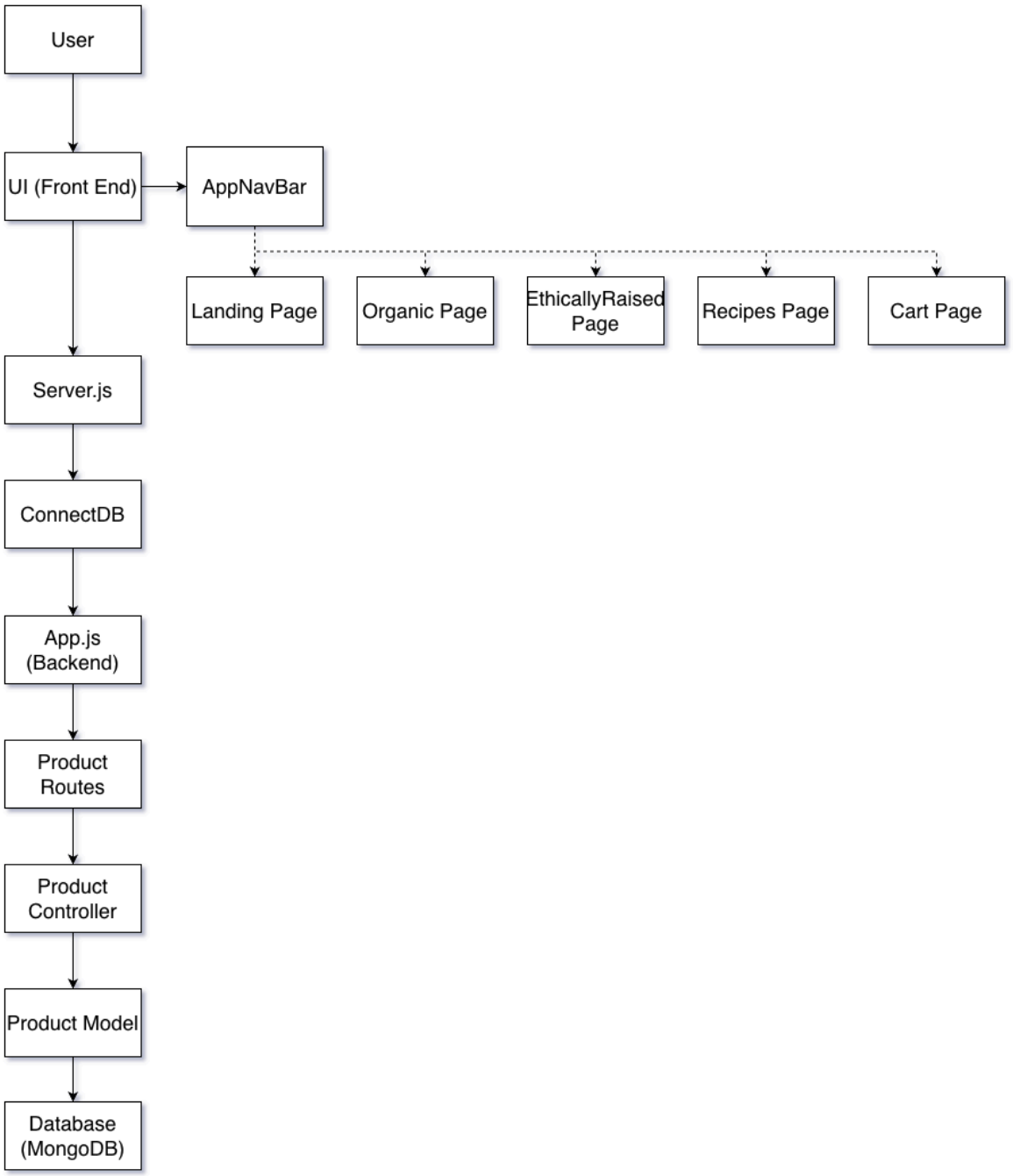
The frontend is implemented as a component-based React single-page application using Vite and styled using **Bootstrap and React-Bootstrap components**. The root component (**App.jsx**) manages **client-side routing based on URL paths** and loads page components, including **Landing, Organic, Ethically Raised, Recipes, and Cart**. These components share common interface elements such as the navigation bar (**AppNavbar**).

Data currently flows from the frontend to the backend through API endpoints such as **/api/products**, which return JSON responses. The frontend currently manages state locally within components without a global state management system.

During development, the backend server runs on **port 4000** while the frontend development server runs on **port 5173**. Environment variables, such as the MongoDB connection string, are configured in a **.env file**.

Chapter 2: System Overview

Architecture Diagram



COMPONENT DESIGN

CRC CARDS

Frontend Components: React Components

AppNavbar	Frontend
Parent Class:	React component
Subclasses:	None
Responsibilities: - Nav links to main sections. - Search/location input. - Cart badge + welcome text.	
Collaborators: - All pages (shared navbar)	

Landing	Frontend
Parent Class:	React component
Subclasses:	None
Responsibilities: - Hero + value props. - CTA to shop (navigate into store pages).	
Collaborators: - AppNavbar	

Organic	Frontend
Parent Class:	React component
Subclasses:	None
Responsibilities: - Tabbed produce grid. - Client-side search/filter. - Product detail modal + CTAs.	
Collaborators: - AppNavbar	

EthicallyRaised — Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Tabbed protein grid.
- Client-side search/filter + modal.

Collaborators:
- AppNavbar

Recipes — Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Tabbed recipe catalog + search.
- Modal showing scaled servings + ingredients.
- CTA to add servings to cart (UI only for now).

Collaborators:
- AppNavbar

Cart — Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Cart view shell (placeholder in Sprint 1).

Collaborators:
- AppNavbar

Backend Components: Node / Express / MongoDB Components

server.js — Backend

Parent Class: None
Subclasses: None

Responsibilities:
- Load env, connect DB, start HTTP server.
- Fail fast if MONGO_URI missing/connection fails.

Collaborators:
- connectDB
- app (Express app)

productRoutes

Backend

Parent Class: Express Router
Subclasses: None

Responsibilities:
- Map /api/products to controller handlers.

Collaborators:
- ProductController

app.js (Express app)

Backend

Parent Class: Express application
Subclasses: None

Responsibilities:
- Configure middleware (JSON parsing, CORS).
- Provide health/root routes.
- Mount product routes.

Collaborators:
- productRoutes

productRoutes

Backend

Parent Class: Express Router
Subclasses: None

Responsibilities:
- Map /api/products to controller handlers.

Collaborators:
- ProductController

ProductController

Backend

Parent Class: None
Subclasses: None

Responsibilities:
- Handle product CRUD (Sprint 1: create + list).
- Return proper HTTP codes (201 success, 400 validation).

Collaborators:
- Product model

Product (model)

Backend

Parent Class: Mongoose Model

Subclasses: None

Responsibilities:

- Define schema + validation (name, price, category, inStock, timestamps).
- Expose queries for controllers (list/sort by createdAt).

Collaborators:

- MongoDB
- ProductController

connectDB

Backend

Parent Class: None

Subclasses: None

Responsibilities:

- Establish Mongo connection and surface errors.

Collaborators:

- Mongoose
- server.js

ASSUMPTION AND FUTURE WORK

The FreshRoute system assumes a development environment that supports **Node.js version 18 or later (Node.js 20+ recommended)** and access to a **MongoDB Community Server**. The backend connects to the database using the `MONGO_URI` environment variable, and the application can run on common operating systems such as **Windows, macOS, or Linux**. During development, the frontend and backend communicate through **HTTP with CORS enabled** to allow requests from the frontend development server. Development typically uses tools such as **VS Code and Git for version control**, though these are not strict requirements.

Error handling and system performance are currently implemented at a basic level. Validation errors return **HTTP 400 responses**, and database connection failures terminate the server after logging an error. The application runs as a **single Node.js process**, which is sufficient for development and course-scale use but may require improvements such as caching or pagination for larger workloads. At the moment, frontend pages primarily display UI components and placeholders, while full API integration for **products, recipes, and cart functionality** will be implemented in later sprints.

Future development may expand the system by introducing **user authentication, improved security measures, and better frontend error feedback**. Additional enhancements may include completing API integration across all pages, improving performance and scalability, and introducing **global state management** to better coordinate data across frontend components.